

Documentation Generale du Projet FreelanceHub

1. Presentation generale

FreelanceHub est une application mobile/web de mise en relation entre deux types d'utilisateurs:

- 1 le `client`, qui publie un projet et recrute un freelancer
- 1 le `freelancer`, qui consulte les projets, postule, discute avec le client et suit l'avancement de la mission

Le projet est compose de deux grandes parties:

- 1 un `frontend` developpe avec `Angular 20 + Ionic 8`
- 1 un `backend` developpe avec `Flask + MongoDB`

L'objectif fonctionnel est de couvrir le cycle complet d'une mission freelance:

- 1 inscription et connexion
- 2 publication d'un projet
- 3 consultation des freelancers et des categories
- 4 candidature sur un projet
- 5 acceptation ou refus de la candidature
- 6 creation d'un accord de mission
- 7 messagerie entre client et freelancer
- 8 suivi d'avancement du projet
- 9 notifications
- 10 favoris

Le projet integre aussi une logique metier importante: essai gratuit pour le premier projet, puis commission de plateforme, ainsi qu'un systeme de plans d'abonnement.

2. Idee et demarche generale

La demarche generale du projet est la suivante:

- 1 construire une place de marche de services freelance
- 1 separer clairement le front et le back
- 1 exposer des API REST cote backend

- 1 consommer ces API cote frontend avec `fetch`
- 1 utiliser `MongoDB` pour stocker des donnees flexibles: utilisateurs, projets, candidatures, messages, favoris, notifications, abonnements

La logique du projet repose sur une architecture simple:

- 1 le `frontend` gere l'affichage, la navigation, les formulaires et l'experience utilisateur
- 1 le `backend` gere les regles metier, la securite, les verifications et l'accès a la base
- 1 la `base MongoDB` stocke l'etat des utilisateurs et des interactions

Cette demarche est adaptee a un projet academique parce qu'elle permet de montrer:

- 1 une architecture moderne en couches
- 1 une authentication par token
- 1 des roles utilisateur
- 1 des operations CRUD
- 1 des flux metier complets
- 1 une base NoSQL avec relations gerees par references

3. Architecture globale

3.1 Frontend

Le frontend se trouve dans `free lancehub-ionic/`.

Technologies:

- 1 `Angular 20`
- 1 `Ionic 8`
- 1 composants `standalone`
- 1 `Angular Router`
- 1 `localStorage` pour conserver la session

Role du frontend:

- 1 afficher les pages
- 1 recuperer les donnees depuis l'API
- 1 envoyer les formulaires au backend
- 1 proteger certaines routes avec un `authGuard`

- 1 stocker localement le token JWT et l'utilisateur connecte

Les principales pages sont:

- 1 `welcome`
- 1 `auth`
- 1 `home`
- 1 `client-request`
- 1 `service-market`
- 1 `project-detail`
- 1 `my-jobs`
- 1 `messages`
- 1 `notifications`
- 1 `favorites`
- 1 `project-progress`
- 1 `edit-profile`
- 1 `freelancer-profile`

3.2 Backend

Le backend se trouve dans `freelancehub-backend/`.

Technologies:

- 1 `Flask`
- 1 `Flask-PyMongo`
- 1 `Flask-JWT-Extended`
- 1 `Flask-CORS`
- 1 `bcrypt`
- 1 `MongoDB Atlas`

Role du backend:

- 1 gerer l'authentification
- 1 appliquer les regles metier
- 1 verifier les roles
- 1 exposer les API REST
- 1 lire et ecrire dans MongoDB
- 1 creer des notifications

Structure backend:

- 1 ``run.py`` : point d'entree de l'application
- 1 ``app/__init__.py`` : factory Flask + initialisation extensions + blueprints
- 1 ``app/config.py`` : configuration
- 1 ``app/middleware/auth.py`` : securite et verification des roles
- 1 ``app/routes/`` : routes API
- 1 ``app/utls/notifications.py`` : creation et serialisation des notifications

3.3 Base de donnees

Le projet utilise MongoDB, une base NoSQL orientee documents.

Pourquoi MongoDB est pertinent ici:

- 1 les profils freelancers ont des champs variables
- 1 les projets contiennent des sous-structures comme ``contract`` et ``progress_entries``
- 1 les relations peuvent etre gerees par ``ObjectId``
- 1 c'est souple pour un projet evolutif

4. Initialisation de l'application

Au demarrage:

- 1 ``run.py`` appelle ``create_app()``
- 2 ``create_app()`` charge la configuration selon l'environnement
- 3 Flask initialise:
 - 1 ``PyMongo``
 - 1 ``JWTManager``
 - 1 ``CORS``
- 1 les blueprints sont enregistres sous le prefixe ``/api``
- 2 une route ``/api/health`` permet de verifier que l'API et MongoDB fonctionnent

Cette approche s'appelle Application Factory Pattern.

Pourquoi c'est bien:

- 1 facilite la maintenance

- 1 facilite les tests
- 1 separe creation de l'app et execution

5. Gestion de l'authentification et des roles

Le projet utilise principalement JWT pour l'authentification.

Fonctionnement:

- 1 l'utilisateur s'inscrit ou se connecte
- 2 le backend cree un `access\_token` et un `refresh\_token`
- 3 le frontend stocke le token dans `localStorage`
- 4 les requetes authentifiees envoient:
 - 1 `Authorization: Bearer <token>`
 - 1 et aussi `X-User-Id`

Le backend sait ensuite quel utilisateur effectue l'action.

Roles geres:

- 1 `client`
- 1 `freelancer`

Decorateurs de protection:

- 1 `token\_required`
- 1 `client\_only`
- 1 `freelancer\_only`

Exemples:

- 1 seul un client peut creer un projet
- 1 seul un freelancer peut postuler a un projet
- 1 seuls les participants d'une conversation peuvent lire ses messages

6. Structure des API backend

Les principaux modules API sont:

- 1 `/api/auth`
- 1 `/api/users`
- 1 `/api/projects`
- 1 `/api/categories`
- 1 `/api/payments`
- 1 `/api/messages`
- 1 `/api/notifications`

6.1 Auth

Fonctions principales:

- 1 inscription
- 1 connexion
- 1 rafraichissement du token

Points techniques:

- 1 validation email
- 1 hash du mot de passe avec `bcrypt`
- 1 verification du role
- 1 creation automatique d'un profil freelancer si le role est `freelancer`

6.2 Users

Fonctions principales:

- 1 recuperer son profil
- 1 modifier son profil
- 1 lister les freelancers
- 1 consulter un freelancer
- 1 voir ses candidatures
- 1 gerer ses favoris
- 1 poster un avis sur un freelancer

Le backend enrichit certains champs pour faciliter l'affichage mobile, par exemple:

- 1 `hourlyRate`
- 1 `completedJobs`

1 `reviews`

6.3 Projects

C'est le coeur metier du projet.

Fonctions principales:

- 1 lister les projets avec filtres et pagination
- 1 creer un projet
- 1 consulter un projet
- 1 modifier et supprimer logiquement un projet
- 1 postuler a un projet
- 1 lister les candidatures
- 1 accepter ou refuser une candidature
- 1 changer le statut du projet
- 1 gerer le contrat
- 1 gerer le suivi d'avancement
- 1 lister les projets du client ou du freelancer

Les statuts principaux d'un projet sont:

- 1 `open`
- 1 `in-progress`
- 1 `completed`
- 1 `cancelled`

6.4 Categories

Fonctions principales:

- 1 lister les categories avec compteurs
- 1 afficher le detail d'une categorie
- 1 creer, modifier, desactiver une categorie

Le module calcule:

- 1 nombre de projets ouverts
- 1 nombre de freelancers disponibles
- 1 budget moyen

6.5 Payments

Ce module simule la logique economique de la plateforme.

Fonctions principales:

- 1 calcul de commission
- 1 paiement d'un projet
- 1 abonnement a un plan
- 1 historique des transactions
- 1 statut de l'abonnement

Logique metier:

- 1 premier projet: `0% commission`
- 1 ensuite: `5% client + 5% freelancer`
- 1 plans possibles: `free-trial`, `basic`, `pro`

6.6 Messages

Fonctions principales:

- 1 lister les conversations
- 1 afficher les messages d'une conversation
- 1 envoyer un message
- 1 creer une nouvelle conversation

Le systeme marque aussi les messages comme lus.

6.7 Notifications

Fonctions principales:

- 1 lister les notifications
- 1 marquer une notification comme lue
- 1 tout marquer comme lu

Les notifications sont creees automatiquement lors d'evenements metier:

- 1 nouvelle candidature
- 1 reponse a une candidature
- 1 nouveau message

- 1 signature de contrat
- 1 avancement du projet

7. Modele de donnees principal

Les principales collections MongoDB identifiees dans le code et dans seed_v2.js sont:

- 1 `users`
- 1 `freelancers`
- 1 `projects`
- 1 `applications`
- 1 `reviews`
- 1 `categories`
- 1 `conversations`
- 1 `messages`
- 1 `notifications`
- 1 `favorites`
- 1 `transactions`
- 1 `subscriptions`
- 1 `plans`

7.1 Collection users

Contient:

- 1 nom
- 1 email
- 1 mot de passe hash
- 1 role
- 1 avatar
- 1 telephone
- 1 indicateur `trial\_used`
- 1 dates de creation et mise a jour

7.2 Collection freelancers

Contient:

- 1 reference `user\_id`
- 1 titre
- 1 bio
- 1 competences
- 1 tarif horaire
- 1 localisation
- 1 disponibilite
- 1 note
- 1 nombre d'avis
- 1 jobs completes
- 1 langues
- 1 education
- 1 experience
- 1 portfolio

7.3 Collection projects

Contient:

- 1 `client\_id`
- 1 titre
- 1 description
- 1 budget min et max
- 1 categorie
- 1 statut
- 1 freelancer accepte
- 1 montant convenu
- 1 historique des statuts
- 1 contrat
- 1 entrees de progression

Le choix de MongoDB est tres utile ici car:

- 1 `contract` est un sous-document
- 1 `progress\_entries` est un tableau de sous-documents
- 1 `status\_history` est un tableau de sous-documents

7.4 Collection applications

Contient:

- 1 `project\_id`
- 1 `client\_id`
- 1 `freelancer\_id`
- 1 montant de l'offre
- 1 commission
- 1 montant net
- 1 lettre de motivation
- 1 statut de candidature

7.5 Collections conversations et messages

conversations contient la liste des participants.

messages contient:

- 1 `conversation\_id`
- 1 `sender\_id`
- 1 texte
- 1 statut de lecture
- 1 date

7.6 Collections économiques

- 1 `plans` : définition des offres
- 1 `subscriptions` : abonnement actif d'un utilisateur
- 1 `transactions` : historique des paiements

8. Fonctionnement du frontend

Le frontend Angular/Ionic fonctionne avec des composants standalone.

8.1 Routage

Le fichier de routage déclare les pages et applique `AuthGuard` sur les pages réservées.

Exemples de routes protégées:

- 1 `/home`

- 1 `/client-request`
- 1 `/my-jobs`
- 1 `/messages`
- 1 `/notifications`
- 1 `/favorites`
- 1 `/project-progress`
- 1 `/edit-profile`

8.2 Gestion de session

Le frontend stocke dans `localStorage`:

- 1 `fh\_token`
- 1 `fh\_user`
- 1 `fh\_api\_base`

Avantage:

- 1 implementation simple
- 1 pratique pour un prototype

Limite:

- 1 `localStorage` est moins securise qu'un stockage via cookies `HttpOnly`

8.3 Communication avec l'API

Le frontend n'utilise pas `HttpClient` Angular, mais `fetch`.

Cela reste valable techniquement, mais il faut savoir l'expliquer:

- 1 `fetch` simplifie la logique
- 1 moins de couche supplementaire
- 1 mais on perd certains avantages Angular comme les interceptors HTTP

8.4 Pages majeures

Home

Affiche:

- 1 categories
- 1 projets ouverts

1 compteurs de notifications et messages

Client Request

Permet au client de:

- 1 publier un projet
- 1 voir ses projets
- 1 rechercher des freelancers

Service Market

Permet:

- 1 filtrage des projets
- 1 filtrage des freelancers
- 1 navigation par categorie

Project Detail

Permet:

- 1 consulter un projet
- 1 postuler
- 1 changer le statut
- 1 gerer le contrat
- 1 gerer la progression
- 1 ajouter en favori

My Jobs

Vue differente selon le role:

- 1 pour le freelancer: ses candidatures
- 1 pour le client: ses projets et les candidats

Messages

Permet la communication directe entre deux utilisateurs.

Edit Profile

Permet de modifier:

- 1 donnees personnelles
- 1 competences
- 1 education
- 1 experience
- 1 portfolio

9. Flux metier principal

Cas 1: creation et traitement d'un projet

- 1 le client se connecte
- 2 il cree un projet
- 3 le projet est stocke dans `projects` avec statut `open`
- 4 les freelancers consultent les projets
- 5 un freelancer postule
- 6 une candidature est enregistree dans `applications`
- 7 le client voit les candidatures
- 8 il accepte une candidature
- 9 le projet passe en `in-progress`
- 10 le freelancer retenu est associe au projet
- 11 les autres candidatures sont rejetees automatiquement

Cas 2: contrat et suivi

- 1 le client redige les termes du contrat
- 2 le contrat est enregistre dans le projet
- 3 le freelancer signe
- 4 le projet peut ensuite etre suivi avec des entrees de progression
- 5 le client voit l'avancement, les heures et les blocages

Cas 3: messagerie

- 1 un utilisateur ouvre une conversation
- 2 si elle n'existe pas, elle est creee
- 3 les messages sont stockes dans `messages`
- 4 le destinataire recoit une notification

Cas 4: favoris

L'utilisateur peut ajouter un:

- 1 projet
- 1 freelancer

Les favoris sont stockés dans une collection séparée, ce qui évite de dupliquer les données.

10. Choix techniques importants à expliquer

10.1 Pourquoi Flask?

- 1 léger
- 1 simple à comprendre
- 1 rapide à mettre en place
- 1 très adapté à une API REST pour projet scolaire

10.2 Pourquoi Angular + Ionic?

- 1 Angular structure bien une application complexe
- 1 Ionic facilite la création d'une interface mobile
- 1 le même code peut viser une expérience proche du mobile

10.3 Pourquoi MongoDB?

- 1 données hétérogènes
- 1 profils freelancers riches et variables
- 1 sous-documents utiles pour contrat et progression
- 1 souplesse de schéma

10.4 Pourquoi JWT?

- 1 authentification stateless
- 1 bien adaptée aux API REST
- 1 facile à utiliser entre frontend et backend séparés

10.5 Pourquoi séparer front et back?

- 1 meilleure maintenance

- 1 responsabilites claires
- 1 possibilite de changer l'un sans casser l'autre
- 1 facilite le travail en equipe

11. Questions techniques que le professeur peut poser

Voici les questions les plus probables, avec des reponses courtes que vous pouvez reprendre a l'oral.

11.1 Question: quelle est l'architecture de votre projet?

Reponse:

Le projet suit une architecture client-serveur. Le frontend Angular/Ionic gere l'interface et le backend Flask expose des API REST. MongoDB stocke les donnees sous forme de documents. Le frontend consomme les endpoints `/api/...` pour afficher ou modifier les donnees.

11.2 Question: pourquoi avoir choisi MongoDB au lieu d'une base relationnelle?

Reponse:

Parce que les profils freelancers, les contrats et le suivi de projet ont des structures flexibles. MongoDB permet de stocker facilement des tableaux et des sous-documents sans schema rigide, par exemple `progress_entries` et `contract`.

11.3 Question: comment gerez-vous l'authentification?

Reponse:

Le backend genere un token JWT a la connexion. Le frontend le stocke dans `localStorage` et l'envoie dans l'en-tete `Authorization`. Le backend verifie ce token pour autoriser l'accès aux routes protegees.

11.4 Question: comment differenciez-vous client et freelancer?

Reponse:

Chaque utilisateur a un champ `role`. Ensuite, le backend applique des decorateurs comme `client_only` ou `freelancer_only` pour restreindre certaines routes.

11.5 Question: comment protege-t-on une route cote frontend?

Reponse:

Avec `AuthGuard`. Si aucun token n'est present, l'utilisateur est redirige vers `/auth`.

11.6 Question: comment protege-t-on une route cote backend?

Reponse:

Avec des decorateurs comme `token_required`, puis des verifications de role et de propriete. Par exemple, un client ne peut modifier qu'un projet qui lui appartient.

11.7 Question: comment fonctionne le cycle d'un projet?

Reponse:

Le projet commence en `open`, puis passe en `in-progress` quand une candidature est acceptee. Ensuite il peut devenir `completed` ou `cancelled`. Un historique de statut est conserve.

11.8 Question: comment gerer les relations dans MongoDB?

Reponse:

Avec des `ObjectId`. Par exemple, un projet contient `client_id`, une candidature contient `project_id` et `freelancer_id`. On fait ensuite des recherches manuelles entre collections.

11.9 Question: pourquoi avez-vous utilise bcrypt?

Reponse:

Pour ne jamais stocker les mots de passe en clair. `bcrypt` hash le mot de passe avec salage, ce qui augmente la securite.

11.10 Question: comment fonctionne la messagerie?

Reponse:

Une collection `conversations` stocke les participants, et une collection `messages` stocke les messages. Quand un message est envoye, une notification est creee pour l'autre utilisateur.

11.11 Question: comment fonctionne le suivi d'avancement?

Reponse:

Le projet contient un tableau `progress_entries`. Chaque entree ajoute un resume, des heures, des taches, un pourcentage et d'éventuels blocages. Le backend recalcule ensuite les indicateurs globaux.

11.12 Question: comment gerez-vous les notifications?

Reponse:

Les notifications sont creees par le backend lors d'évenements importants, puis listees dans une collection dediee. Le frontend peut les afficher et les marquer comme lues.

11.13 Question: comment gerez-vous les favoris?

Reponse:

On ne modifie pas directement les documents projets ou freelancers. On cree une collection `favorites` avec `user_id`, `entity_type` et `entity_id`.

11.14 Question: comment fonctionne le systeme de commission?

Reponse:

Le premier projet peut etre gratuit via `trial_used = false`. Ensuite le backend applique une commission. Les montants client, freelancer et revenu plateforme sont calcules cote serveur.

11.15 Question: pourquoi le calcul de commission doit-il etre fait dans le backend?

Reponse:

Parce que la logique metier ne doit pas dependre du frontend. Si on la laisse seulement cote client, elle serait facile a contourner.

11.16 Question: comment gerez-vous la pagination?

Reponse:

Le backend lit `page` et `page_size`, applique `skip` et `limit`, puis renvoie aussi `total` et `pages`.

11.17 Question: quelle difference entre validation frontend et backend?

Reponse:

Le frontend valide pour l'experience utilisateur. Le backend valide pour la securite et l'integrite des donnees. Meme si le frontend est contourné, le backend doit rester correct.

11.18 Question: pourquoi avez-vous choisi une API REST?

Reponse:

Parce qu'elle est simple, standard, lisible et bien adaptee a la communication entre une app Angular/Ionic et un backend Flask.

11.19 Question: qu'est-ce qu'un blueprint Flask?

Reponse:

C'est un moyen de modulariser les routes. Au lieu d'avoir un seul gros fichier, on separe les routes par domaine: users, projects, messages, payments, etc.

11.20 Question: qu'est-ce que l'Application Factory?

Reponse:

C'est un pattern Flask ou l'application est creee dans une fonction `create_app()`. Cela rend le projet plus propre et plus facile a configurer selon les environnements.

11.21 Question: pourquoi avoir utilise des composants standalone dans Angular?

Reponse:

Parce que c'est la direction moderne d'Angular. Cela allège la structure et évite de multiplier les modules classiques.

11.22 Question: pourquoi utilisez-vous fetch et pas HttpClient?

Reponse:

Pour garder une implementation simple et directe. C'est suffisant pour ce projet, meme si HttpClient serait plus evolutif pour une grande application.

11.23 Question: comment assurez-vous qu'un freelancer ne peut pas postuler deux fois?

Reponse:

Le backend verifie s'il existe deja une candidature pour ce project_id et ce freelancer_id avant d'insérer une nouvelle candidature.

11.24 Question: comment assurez-vous qu'un utilisateur ne voit que ses messages?

Reponse:

Le backend verifie que son ObjectId est bien present dans participants de la conversation.

11.25 Question: quelle est la difference entre users et freelancers?

Reponse:

users contient les informations communes a tous les comptes. freelancers contient les informations specialisees du role freelancer comme les competences, le portfolio et le tarif horaire.

12. Points forts du projet

- 1 architecture separee frontend/backend
- 1 gestion de roles utilisateur
- 1 authentication JWT
- 1 API REST bien decoupee
- 1 fonctionnalites metier riches
- 1 gestion des candidatures
- 1 contrat de mission
- 1 suivi de progression
- 1 messagerie
- 1 notifications
- 1 favoris
- 1 logique economique avec commission et abonnement

13. Limites techniques et points a assumer devant le professeur

Il est tres important de ne pas presenter le projet comme parfait. Dire les limites montre de la maturite technique.

13.1 Securite perfectible

Points sensibles observes:

- 1 des identifiants MongoDB sont ecrits en dur dans le code de configuration et dans le seed
- 1 le frontend stocke le JWT dans `localStorage`
- 1 le backend accepte aussi `X-User-Id`, ce qui simplifie les tests mais est moins strict qu'une authentication JWT pure

Ce que vous pouvez dire:

Pour un projet academique, cela a accelere le developpement. En production, il faudrait deplacer tous les secrets dans des variables d'environnement, supprimer la dependance a X-User - Id et utiliser une gestion de session plus securisee.

13.2 Absence d'index MongoDB explicites

Le code ne cree pas d'index pour les recherches frequentes.

Impact:

- 1 performances moins bonnes si les donnees grandissent

Amelioration:

- 1 index sur `email`, `project\_id`, `freelancer\_id`, `participants`, `user\_id`, `status`

13.3 Pas de couche service frontend

Le frontend appelle fetch directement dans les pages.

Impact:

- 1 code plus rapide a ecrire
- 1 mais moins reutilisable et moins testable

Amelioration:

- 1 centraliser les appels dans des services Angular

13.4 Pas de tests automatises visibles

Le projet ne montre pas de suite de tests backend ou frontend.

Amelioration:

- 1 tests unitaires
- 1 tests d'integration API
- 1 tests de composants Angular

13.5 Transactions metier simplifiees

Certaines operations touchent plusieurs collections sans transaction MongoDB explicite.

Impact:

- 1 risque d'incoherence en cas d'erreur en plein milieu

Amelioration:

- 1 utiliser des transactions MongoDB si le cluster et le contexte le permettent

14. Ce que le professeur peut demander comme ameliorations futures

- 1 ajouter un vrai systeme de paiement en ligne
- 1 ajouter un chat temps reel avec WebSocket
- 1 ajouter une authentication plus securisee
- 1 ajouter des tests automatises
- 1 ajouter un tableau de bord admin
- 1 ajouter des index MongoDB
- 1 ajouter une couche service cote Angular
- 1 ajouter la gestion des fichiers et pieces jointes
- 1 ajouter des appels video

15. Conclusion

FreelanceHub est un projet complet de marketplace freelance qui montre:

- 1 une application front moderne en Angular/Ionic
- 1 une API REST backend en Flask
- 1 une base MongoDB adaptee aux donnees flexibles
- 1 une vraie logique metier avec roles, projets, candidatures, contrats, progression, messagerie, notifications et paiements

Le projet est interessant pedagogiquement parce qu'il ne se limite pas a du CRUD simple. Il met en oeuvre plusieurs concepts importants du developpement full stack:

- 1 architecture client-serveur
- 1 authentication
- 1 gestion des roles
- 1 modelisation NoSQL
- 1 orchestration de flux metier
- 1 experience utilisateur mobile

Si vous presentez ce projet a l'oral, il faut insister sur:

- 1 la separation claire entre frontend et backend
- 1 le cycle de vie complet d'un projet freelance
- 1 la logique de commission et d'abonnement
- 1 les choix techniques faits pour aller vite tout en restant structure
- 1 les limites connues et les ameliorations possibles

16. Resume ultra-court pour l'oral

Si le professeur demande un resume rapide, vous pouvez dire:

Notre projet est une plateforme freelance full stack. Le frontend est en Angular/Ionic et le backend en Flask avec MongoDB. Un client peut publier un projet, un freelancer peut postuler, le client peut accepter une candidature, creer un contrat, suivre l'avancement, discuter via messagerie et recevoir des notifications. Nous avons aussi implemente une logique metier de commission et d'abonnement.